

Pertemuan 8: Interface dan Abstract Class dalam Dart

1. Identitas Pembelajaran

Keterangan	Detail
Mata Kuliah	Pemrograman Berorientasi Objek (PBO)
SKS	3 SKS (1 Teori, 2 Praktik)
Semester	3
Pertemuan	8

2. Tujuan Pembelajaran

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

- 1. Memahami perbedaan antara *abstract class* (kontrak parsial) dan *class* konkret (biasa).
- 2. Mendefinisikan *abstract method* — metode tanpa implementasi yang wajib diisi oleh *child class*.
- 3. Memahami konsep *interface* sebagai kontrak penuh dan cara Dart mengimplementasikannya.
- 4. Membedakan penggunaan *extends* (pewarisan) vs *implements* (kontrak antarmuka).
- 5. Menggunakan *multiple interface* — satu *class* mengimplementasikan lebih dari satu kontrak.

3. Rencana Aktivitas Pembelajaran (3 x 50 Menit)

Waktu	Aktivitas Dosen	Aktivitas Mahasiswa	Target Capaian
00 - 30 Menit	Recall P7. Menjelaskan kebutuhan <i>abstract class</i> — kontrak yang wajib diikuti <i>child class</i> .	Mengamati error saat mencoba membuat objek dari <i>abstract class</i> .	Memahami tujuan <i>abstract class</i> .
30 - 80 Menit	Live Coding: <i>abstract class</i> , <i>abstract method</i> , <i>extends</i> pada <i>abstract class</i> , dan menunjukkan perbedaan dengan <i>implements</i> .	Mengikuti <i>coding</i> , mencoba membuat <i>class</i> tanpa mengimplementasikan <i>abstract method</i> dan mengamati error.	Mampu membuat dan menggunakan <i>abstract class</i> .
80 - 120 Menit	Menjelaskan dan mendemonstrasikan <i>interface</i> di Dart dan <i>multiple interface</i> .	Membuat <i>class</i> yang <i>implements</i> dua <i>interface</i> berbeda sekaligus secara mandiri.	Memahami <i>multiple interface inheritance</i> .
120 - 150 Menit	Evaluasi singkat: kapan pakai <i>extends</i> vs <i>implements</i> vs <i>abstract</i> ?	Diskusi terbuka.	Penguatan konsep desain OOP.

4. Abstract Class

A. Apa itu Abstract Class?

Abstract Class adalah *class* yang **tidak bisa diinstansiasi secara langsung** — artinya, kita tidak bisa membuat objek dari *abstract class* menggunakan `NamaClass()`. *Abstract class* hanya bisa digunakan sebagai *parent class* yang diwarisi oleh *class* lain.

Gunakan kata kunci `abstract` sebelum `class`.

```
abstract class Animal {
    String name;
    Animal(this.name);

    void breathe() => print('$name sedang bernapas.');// Method biasa – boleh punya isi
    void makeSound();// Abstract method – tanpa isi, wajib diimplementasikan child
}

void main() {
    // Animal a = Animal('Hewan');// ERROR! Abstract class tidak bisa diinstansiasi.
    // "Abstract classes can't be instantiated."
}
```

B. Kapan Menggunakan Abstract Class?

Abstract class digunakan ketika kita ingin:

- Mendefinisikan **template** atau **kerangka** yang wajib diikuti oleh semua *child class*.
- Menyediakan beberapa *method* yang sudah **diimplementasikan** (bisa diwarisi langsung).
- Sekaligus mewajibkan *child class* untuk mengimplementasikan *method* tertentu sendiri.

5. Abstract Method

Abstract Method adalah *method* yang dideklarasikan di dalam *abstract class* **tanpa badan** (tanpa blok `{ }`). Semua *child class* yang mewarisi *abstract class* tersebut **wajib** mengimplementasikan setiap *abstract method*-nya, atau akan terjadi error saat kompilasi.

```
abstract class Shape {
    String color;
    Shape(this.color);

    // Abstract method – tanpa implementasi. Child class WAJIB mengisi ini.
    double calculateArea();
    double calculatePerimeter();

    // Concrete method – sudah ada implementasinya, bisa diwarisi langsung
}
```

```
void describe() {
    print('Bentuk berwarna $color');
    print('  Luas      : ${calculateArea().toStringAsFixed(2)} cm²');
    print('  Keliling : ${calculatePerimeter().toStringAsFixed(2)} cm');
}

class Circle extends Shape {
    double radius;
    Circle(String color, this.radius) : super(color);

    @override
    double calculateArea() => 3.14159 * radius * radius;

    @override
    double calculatePerimeter() => 2 * 3.14159 * radius;
}

class Rectangle extends Shape {
    double width, height;
    Rectangle(String color, this.width, this.height) : super(color);

    @override
    double calculateArea() => width * height;

    @override
    double calculatePerimeter() => 2 * (width + height);
}

class Triangle extends Shape {
    double a, b, c; // Tiga sisi segitiga
    Triangle(String color, this.a, this.b, this.c) : super(color);

    @override
    double calculateArea() {
        // Rumus Heron's Formula
        double s = (a + b + c) / 2;
        return (s * (s - a) * (s - b) * (s - c));
    }

    @override
    double calculatePerimeter() => a + b + c;
}

void main() {
    List<Shape> shapes = [
        Circle('Merah', 7.0),
        Rectangle('Biru', 8.0, 5.0),
        Triangle('Hijau', 3.0, 4.0, 5.0),
    ];

    for (var shape in shapes) {
        shape.describe();
        print('');
    }
}
```

```
}
}
```

Output:

```
Bentuk berwarna Merah
  Luas      : 153.94 cm²
  Keliling  : 43.98 cm

Bentuk berwarna Biru
  Luas      : 40.00 cm²
  Keliling  : 26.00 cm

Bentuk berwarna Hijau
  Luas      : 6.00 cm²
  Keliling  : 12.00 cm
```

6. Interface di Dart

A. Konsep Dasar Interface

Interface adalah sebuah **kontrak** — daftar *method* dan *field* yang **wajib** dimiliki oleh setiap *class* yang menggunakannya.

Di Dart, **tidak ada kata kunci `interface`** khusus seperti di Java. Sebagai gantinya, **setiap *class* di Dart secara otomatis juga berfungsi sebagai *interface***. Ketika kita menggunakan **`implements`**, kita mengambil "kontrak" dari *class* tersebut — semua *method* dan *field* wajib dideklarasikan ulang.

Anggap saja *interface* adalah kontrak pembuatan *class*.

B. `extends` vs `implements`

Aspek	<code>extends</code>	<code>implements</code>
Hubungan	IS-A (<i>adalah sebuah</i>) — relasi keturunan	CAN-DO (<i>bisa melakukan</i>) — kontrak kemampuan
Field & Method	Diwarisi otomatis — tidak perlu ditulis ulang	Wajib dideklarasikan ulang semuanya
Jumlah parent	Hanya 1	Bisa lebih dari satu sekaligus
Constructor	Diwariskan (via <code>super</code>)	Tidak — harus buat sendiri
Tujuan	Berbagi kode (code reuse)	Menyeragamkan antarmuka (standardisasi)

```
class Kendaraan {
    String platNomor;
    Kendaraan(this.platNomor);

    void bunyikanKlakson() => print('Tiin tiin!');
}

// EXTENDS: mewarisi field dan method secara langsung
class Mobil extends Kendaraan {
    Mobil(String plat) : super(plat);
    // Otomatis punya platNomor dan bunyikanKlakson()
}

// IMPLEMENTS: hanya mengambil "kontrak" – semua wajib ditulis ulang
class Motor implements Kendaraan {
    @override
    String platNomor; // Wajib dideklarasikan sendiri
    Motor(this.platNomor);

    @override
    void bunyikanKlakson() => print('Tin tin!'); // Wajib diimplementasikan sendiri
}
```

7. Membuat dan Menggunakan Interface

A. Interface Sederhana

```
// Class ini berfungsi sebagai interface
class Printable {
    void printDocument() {} // Isi kosong atau langsung pakai abstract class
}

// Lebih baik gunakan abstract class untuk interface
abstract class Printable {
    void printDocument();
}

class InvoicePrinter implements Printable {
    @override
    void printDocument() => print('Mencetak invoice...');
}

class ReportPrinter implements Printable {
    @override
    void printDocument() => print('Mencetak laporan bulanan...');
}
```

B. Interface sebagai Abstract Class (Best Practice)

Dalam praktik, sebaiknya *interface* dibuat sebagai **abstract class** agar lebih jelas bahwa *class* ini adalah kontrak dan tidak bisa diinstansiasi.

```
abstract class PaymentGateway {
    String get name;           // Getter wajib
    void processPayment(double amount);
    void refund(double amount);
    bool verifyTransaction(String transactionId);
}

class GoPayGateway implements PaymentGateway {
    @override
    String get name => 'GoPay';

    @override
    void processPayment(double amount) {
        print('[$name] Memproses Rp$amount via dompet digital.');
```

```
    }

    @override
    void refund(double amount) {
        print('[$name] Refund Rp$amount ke saldo GoPay.');
```

```
    }

    @override
    bool verifyTransaction(String id) {
        print('[$name] Memverifikasi transaksi $id...');
        return true;
    }
}

class BankTransferGateway implements PaymentGateway {
    @override
    String get name => 'Transfer Bank';

    @override
    void processPayment(double amount) {
        print('[$name] Mengirim Rp$amount ke rekening tujuan.');
```

```
    }

    @override
    void refund(double amount) {
        print('[$name] Refund Rp$amount membutuhkan 3-5 hari kerja.');
```

```
    }

    @override
    bool verifyTransaction(String id) {
        print('[$name] Menunggu konfirmasi transfer untuk $id...');
        return true;
    }
}
```

```
void main() {  
    List<PaymentGateway> gateways = [GoPayGateway(), BankTransferGateway()];  
  
    double tagihan = 150000;  
    for (var gw in gateways) {  
        gw.processPayment(tagihan);  
    }  
}
```

8. Multiple Interface Inheritance

Keunggulan utama **implements** dibanding **extends** adalah kemampuan mengimplementasikan **lebih dari satu interface** sekaligus.

- Dengan **extends**, sebuah *class* hanya bisa mewarisi **satu parent class**.
- Dengan **implements**, sebuah *class* bisa mengikuti kontrak dari **banyak class** sekaligus.

```
abstract class Flyable {  
    void fly();  
    double get maxAltitude;  
}  
  
abstract class Swimmable {  
    void swim();  
    double get maxDepth;  
}  
  
abstract class Walkable {  
    void walk();  
    double get walkSpeed;  
}  
  
// Duck bisa terbang, berenang, DAN berjalan – multiple interface  
class Duck implements Flyable, Swimmable, Walkable {  
    String name;  
    Duck(this.name);  
  
    @override double get maxAltitude => 100.0;  
    @override double get maxDepth => 2.0;  
    @override double get walkSpeed => 3.0;  
  
    @override  
    void fly() => print('$name terbang hingga ${maxAltitude}m.');  
    @override  
    void swim() => print('$name berenang hingga kedalaman ${maxDepth}m.');  
    @override  
    void walk() => print('$name berjalan ${walkSpeed} km/jam.');}
```

```
// Submarine hanya perlu Swimmable
class Submarine implements Swimmable {
    @override
    double get maxDepth => 500.0;

    @override
    void swim() => print('Kapal selam menyelam hingga ${maxDepth}m.');
```

```
}

void main() {
    Duck donald = Duck('Donald');
    donald.fly();
    donald.swim();
    donald.walk();

    print('---');

    Submarine sub = Submarine();
    sub.swim();

    // Polymorphism via interface
    List<Swimmable> aquaticThings = [donald, sub];
    print('\n--- Semua yang bisa berenang ---');
    for (var s in aquaticThings) {
        s.swim();
    }
}
```

9. extends + implements Bersamaan

Sebuah *class* bisa sekaligus mewarisi satu *parent class* lewat **extends** **dan** mengimplementasikan satu atau lebih *interface* lewat **implements**.

```
abstract class Serializable {
    String toJson();
    Map<String, dynamic> toMap();
}

abstract class Comparable<T> {
    int compareTo(T other);
}

class BaseEntity {
    int id;
    DateTime createdAt;
    BaseEntity(this.id : createdAt = DateTime.now());
}

class Product extends BaseEntity implements Serializable, Comparable<Product> {
```



```

String name;
double price;

Product(int id, this.name, this.price) : super(id);

@override
String toJson() => '{"id":$id,"name":"$name","price":$price}';

@override
Map<String, dynamic> toMap() => {'id': id, 'name': name, 'price': price};

@override
int compareTo(Product other) => price.compareTo(other.price);

@override
String toString() => 'Product($name: Rp$price)';
}

void main() {
  List<Product> products = [
    Product(3, 'Headphone', 350000),
    Product(1, 'Mouse', 150000),
    Product(2, 'Keyboard', 250000),
  ];

  // Menggunakan Comparable untuk sorting
  products.sort((a, b) => a.compareTo(b));

  print('=== Produk (terurut berdasarkan harga) ===');
  for (var p in products) {
    print('${p.name}: ${p.toJson()}');
  }
}

```

10. Abstract Class vs Interface — Perbandingan Akhir

Aspek	abstract class (extends)	Interface / abstract class (implements)
Instansiasi	Tidak bisa dibuat objek langsung	Tidak bisa dibuat objek langsung
Field	Bisa punya field dengan nilai default	Field wajib dideklarasikan ulang di child
Concrete Method	Bisa punya method berbodi (diwarisi child)	Semua method wajib diimplementasikan ulang
Jumlah parent	Hanya satu	Bisa banyak sekaligus
Cocok untuk	Template dengan beberapa implementasi bawaan	Standar kontrak kemampuan lintas hierarki

11. Rangkuman

Konsep	Keterangan	Sintaks
abstract class	Class yang tidak bisa diinstansiasi, hanya bisa diwarisi.	<code>abstract class Nama { }</code>
abstract method	Method tanpa badan — wajib diimplementasikan child class.	<code>void method();</code>
implements	Mengikuti kontrak interface — semua field & method wajib ditulis ulang.	<code>class A implements B { }</code>
Multiple Interface	Satu class bisa implements banyak interface.	<code>class A implements B, C, D { }</code>
extends + implements	Bisa digunakan bersamaan.	<code>class A extends B implements C { }</code>

12. Tugas Praktikum Mandiri (Asinkronus — 180 Menit)

Studi Kasus: Sistem Ekspedisi Pengiriman Barang

Tujuan: Menggunakan *abstract class* dan *interface* untuk menyeragamkan antarmuka beberapa layanan pengiriman.

Spesifikasi Proyek:

- Buat proyek baru:** `dart create ekspedisi_NIM`
- Buat `abstract class ShippingService`** sebagai kontrak:
 - Abstract getter* `String serviceName`.
 - Abstract method* `double calculateShippingCost(double weightKg, String destination)`.
 - Abstract method* `int estimateDeliveryDays(String destination)`.
 - Concrete method* `void printEstimate(double weightKg, String destination)` yang memanggil dua abstract method di atas.
- Buat `abstract class TrackableService`:**
 - Abstract method* `String trackPackage(String trackingNumber)`.
- Buat 3 layanan ekspedisi:**
 - `JNEService implements ShippingService, TrackableService`: ongkir `weightKg * 12000`, dalam kota 1 hari, luar kota 3 hari.
 - `SiCepatService implements ShippingService, TrackableService`: ongkir `weightKg * 10000`, dalam kota 1 hari, luar kota 2 hari.
 - `PosIndonesiaService implements ShippingService`: ongkir `weightKg * 7000`, dalam kota 2 hari, luar kota 7 hari (tidak ada tracking).
 - "Dalam kota" jika `destination == 'Jakarta'`, selain itu "luar kota".

5. **Buat class `ShippingComparator`** yang menerima `List<ShippingService>` dan memiliki *method* `void compareAll(double weight, String destination)` yang memanggil `printEstimate()` semua layanan.
6. **Di `main()`:** buat `ShippingComparator` berisi ketiga layanan, panggil `compareAll(2.5, 'Surabaya')`. Juga tes fitur tracking pada layanan yang mendukungnya.
7. **Pengumpulan Tugas:** Jalankan dengan `dart run`, *screenshot* terminal, kumpulkan ke LMS.